



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Программное обеспечение ЭВМ и информационные технологии

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Метод построения синтаксических деревьев для
ограниченного естественного русского языка на основе
наиболее употребимых грамматик

Студент ИУ7-84Б

(подпись)

И. С. Бугаков

Руководитель ВКР

(подпись)

Ю. В. Строганов

2026 год

УТВЕРЖДАЮ

Заведующий кафедрой ИУ7

И. В. Рудаков

23.01.2026 г.

З А Д А Н И Е

на выполнение выпускной квалификационной работы

Студент группы ИУ7-74Б

Бугаков Иван Сергеевич

Тема выпускной квалификационной работы

**Метод построения синтаксических деревьев для ограниченного естественного
русского языка на основе наиболее употребимых грамматик**

При выполнении ВКР:

	Используются / Не используются	Да/Нет
1)	Литературные источники и документы, имеющие гриф секретности	Нет
2)	Литературные источники и документы, имеющие пометку «Для служебного пользования», иные пометки, запрещающие открытое опубликование	Нет
3)	Служебные материалы других организаций	Нет
4)	Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана	Нет
5)	Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема квалификационной работы утверждена распоряжением по факультету:	
Название факультета:	«Информатика и системы управления», ИУ
Дата и рег. номер распоряжения:	

Часть 1. Аналитический раздел

Провести анализ предметной области автоматизированного построения синтаксических деревьев ограниченного естественного русского языка. Провести сравнительный анализ существующих методов построения синтаксических деревьев для ограниченного естественного русского языка. Сформулировать требования к разрабатываемому методу. Формализовать постановку задачи в виде IDEF0-диаграммы.

Часть 2. Конструкторский раздел

Разработать метод построения синтаксических деревьев для ограниченного естественного русского языка. Сформулировать ограничения предметной области. Описать и обосновать выбор используемых структур данных. Изложить ключевые этапы метода в виде схем алгоритмов.

Часть 3. Технологический раздел

Обосновать выбор программных средств реализации предложенного метода. Разработать программное обеспечение, реализующее описанный метод. Описать формат входных и выходных данных, пользовательский интерфейс. Обозначить программные реализации наиболее важных алгоритмов, используемых в предложенном методе построения синтаксических деревьев.

Часть 4. Исследовательский раздел

Привести примеры построения синтаксических деревьев с помощью разработанного метода. Определить зависимость времени построения синтаксических деревьев от количества слов в предложении. Графически визуализировать полученные результаты. Оценить точность и полноту разработанного метода.

Оформление квалификационной работы:

Расчетно-пояснительная записка на 50-80 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, слайды и т.п.):

Презентация к выпускной квалификационной работе на 12-20 слайдах.

Дата выдачи задания: «23» января 2026 года.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до «28» мая 2026 года.

Руководитель ВКР	_____	<u>Строганов Ю. В.</u>	<u>23.01.2026 г.</u>
Студент	_____	<u>Бугаков И. С.</u>	<u>23.01.2026 г.</u>

РЕФЕРАТ

Расчетно-пояснительная записка 21 с., 6 рис., 1 таблиц, 105 источников, 1 приложение.

Ключевые слова: КОРПУСНАЯ ЛИНГВИСТИКА, КОМПЬЮТЕРНАЯ ЛИНГВИСТИКА, СИСТЕМА УПРАВЛЕНИЯ КОРПУСНЫМИ ДАННЫМИ, ПАРАЛЛЕЛЬНЫЙ КОРПУС, РАЗМЕТКА, АНАФОРА, МОРФОЛОГИЯ, СИНТАКСИЧЕСКОЕ ДЕРЕВО.

Объект исследования — система управления корпусными данными.

Цель работы — сравнительный анализ существующих методов построения разметок текстов и систем управления корпусными данными.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	7
1 Аналитический раздел	9
1.1 Анализ предметной области	9
1.2 Системы синтаксической разметки	10
1.2.1 Системы зависимостей	10
1.2.2 Системы составляющих	11
1.2.3 Системы синтаксических групп	13
1.2.4 Сравнение систем синтаксической разметки	14
1.3 Методы построения систем составляющих	15
1.3.1 Алгоритм Кока–Янгера–Касами	15
1.3.2 Алгоритм Эрли	17
1.4 Постановка задачи	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19
ПРИЛОЖЕНИЕ А	21

ВВЕДЕНИЕ

Корпусная лингвистика как раздел компьютерной лингвистики занимается разработкой общих принципов построения и использования лингвистических корпусов с применением компьютерных технологий [1].

В настоящее время наблюдается значительный рост объема переводных научно-технических текстов, что свидетельствует о необходимости, с одной стороны, разработки систем автоматического и автоматизированного перевода, а с другой стороны, проведения работ по унификации и стандартизации национальной терминологии. Отсутствие упорядоченных коллекций научно-технических текстов и созданных на их основе терминологических баз данных и знаний существенно тормозит развитие и совершенствование средств искусственного интеллекта по автоматической обработке научно-технических текстов [1].

Анализ современных параллельных корпусов показывает, что они создаются лингвистами вручную или с использованием ограниченного количества средств разметки текстов, что требует значительных временных затрат на выполнение рутинных процедур разметки и выравнивания параллельных текстов. Эта отрасль является недостаточно формализованной, слабо автоматизированной, существующие методы работы не универсальны, а операции по разметке научно-технических текстов выполняются лингвистами собственноручно отдельно для каждого вида разметки. Следовательно, с целью автоматизировать и ускорить процедуру автоматической обработки научно-технических текстов при создании параллельных корпусов необходима инструментальная поддержка процедуры обработки параллельных научно-технических текстов [1].

Современные системы управления корпусными данными позволяют решать широкий спектр задач, связанных с компьютерной лингвистикой. Часто такие системы основаны на готовых решениях, что приводит к проблемам со скоростью выполнения поисковых запросов, гибкостью, масштабируемостью и расширяемостью системы [2].

Параллельные корпуса находят применение в различных областях: в лексикографии при создании электронных многоязычных словарей и баз данных аннотированных переводных соответствий [3], в переводческой деятельности для анализа переводческих соответствий, терминологической согласованности и изучения переводческих стратегий [4], а также в качестве инструмента переводчика научно-технической литературы [5].

Целью данной работы является сравнительный анализ существующих методов построения разметок текстов и систем управления корпусными данными.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- проанализировать предметную область использования параллельных корпусов в корпусной лингвистике, ввести основные понятия, привести классификацию корпусов текстов;
- перечислить методы или группы методов построения разметки текстов;
- сформулировать критерии и сравнить перечисленные методы построения разметки;

— сформулировать критерии и сравнить существующие системы управления корпусными данными.

1 Аналитический раздел

1.1 Анализ предметной области

Ограниченный естественный язык – это подмножество естественного языка, текст на котором, без каких-либо усилий, воспринимается носителем исходного естественного языка, а также не требует длительного изучения для приобретения навыков составления текстов на этом языке, однако обладает сокращенным набором лексики и грамматики. Использование ограниченного естественного языка позволяет снизить время обработки естественно-языковых конструкций, без снижения уровня функциональности [6].

Разметка текста заключается в приписывании словам или предложениям меток (тэгов) в соответствии с характером разметки: морфологические, синтаксические, семантические и другие [7].

Джеффри Лич приводит следующие принципы, которым должна подчиняться разметка общедоступного корпуса [8]:

- разметка должна основываться на доступной для пользователя в виде руководства или инструкции схеме анализа, в которой должно быть мотивированно введение каждого параметра;
- разметка общедоступного корпуса должна опираться на общеизвестную систему правил и классификаций;
- должно быть ясно кто и как разрабатывает схему аннотации и каковы ограничения, например юридические или технические, при использовании корпусом [9].

Синтаксическая разметка является результатом разбора, выполняемого на основе данных морфологического анализа. Данный вид разметки описывает синтаксические связи и конструкции, образованные лексическими единицами [7, 9]. Тексты, получившие синтаксическую разметку известны в англоязычной литературе как treebanks [10]. Пример визуализации синтаксических зависимостей, отображаемых синтаксической разметкой, представлен на рисунке 1.1.

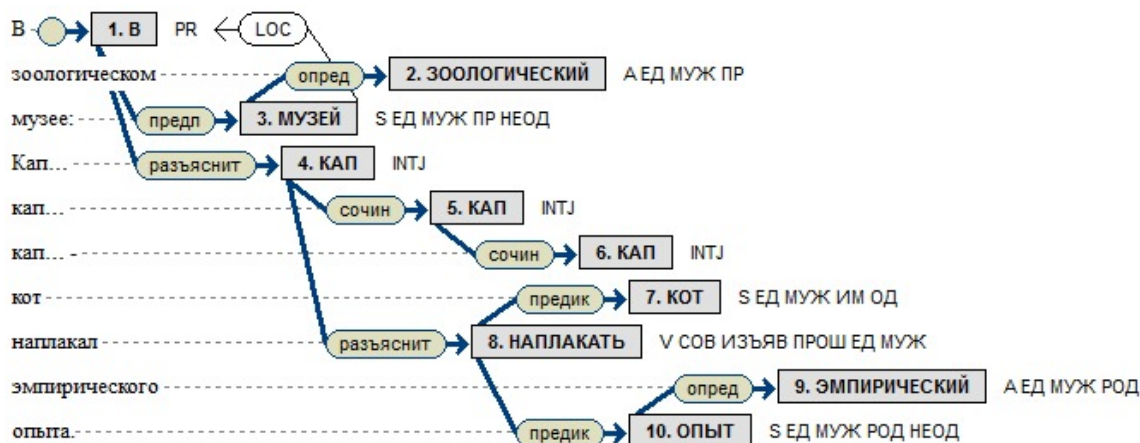


Рисунок 1.1 — Визуализация синтаксических зависимостей в НКРЯ

В корпусной лингвистике наблюдается большое разнообразие синтаксических теорий и формализмов, среди которых наиболее распространенными являются следующие [7, 11]:

- системы зависимостей;
- системы непосредственных составляющих;
- системы синтаксических групп.

1.2 Системы синтаксической разметки

1.2.1 Системы зависимостей

Если Π — конечное линейное упорядоченное множество, то всякое дерево D , для которого Π служит множеством узлов, называется деревом синтаксического подчинения (зависимостей) на Π . Если Π — множество точек некоторой цепочки x , говорят, что D — дерево синтаксического подчинения (зависимостей) для x [11]. Размеченным деревом подчинения(зависимостей) называется упорядоченная четверка 1.1:

$$\langle \Pi, \rightarrow, Z, \psi \rangle, \quad (1.1)$$

где $\langle \Pi, \rightarrow \rangle$ — дерево зависимостей на Π ,

Z — конечное множество меток,

ψ — разметка, отображение множества дуг дерева в Z .

Таким образом дерево зависимостей представляет собой дерево, в котором вершины соответствуют отдельным словам предложения, а дуги между ними — синтаксическим подчинительным связям между словами [12]. Пример дерева зависимостей для шведского языка приведен на рисунке 1.2.

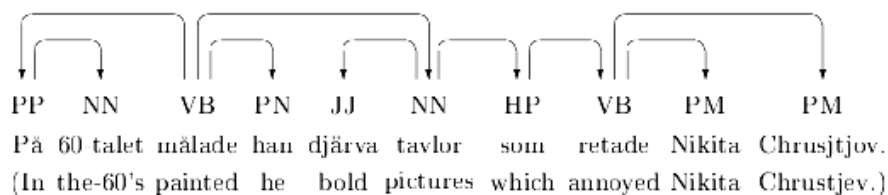


Рисунок 1.2 — Дерево зависимостей предложения на шведском языке

Дерево подчинения $\langle \Pi, \rightarrow \rangle$ называется проективным, если для любых трех его узлов α, β, γ из того, что $\alpha \rightarrow \beta$ и γ лежит между α и β , следует, что γ зависит от α . Проективные предложения соответствуют «естественному» порядку слов, тогда как непроективные более характерны художественной литературе [11]. Например, следующий оборот не является проективным: «... рассмотрев выдвинутые здесь предложения товарищем Лопаткиным...».

Требование проективности исходного предложения и, как следствие, получаемого дерева

зависимостей, является необходимым для большинства алгоритмов построения деревьев зависимостей. Алгоритм Нивре [13], являющийся основой MaltParser [14], алгоритм, используемый в LaPaRus [15] также основываются на предположении, что предложение проективно.

Среди преимуществ систем зависимостей выделяют [11, 13, 16]:

- алгоритм, предложенный Нивре, позволяет построить дерево зависимостей за $O(n)$, где n — число слов в предложении;
- независимость от порядка слов в предложении;
- однозначность отношения «главное–зависимое» между словоформами.

Помимо требования проективности к недостаткам систем зависимостей относят [11, 15, 16, 17]:

- невозможность отобразить связи и иерархию словосочетаний;
- невозможность в полной мере описать неподчинительные связи;
- неоднозначность определения управляющей словоформы в некоторых случаях;
- невозможность различить адьюнкты и аргументы при предикате.

1.2.2 Системы составляющих

Множество C отрезков конечного линейно упорядоченного множества Π называется системой составляющих на Π , если оно удовлетворяет следующим условиям [11]:

- 1) C содержит в качестве элементов само Π и все одноточечные отрезки Π ;
- 2) любые два отрезка из C либо не пересекаются, либо один из них содержится в другом.

Если Π — множество точек некоторой цепочки x , говорят о системе составляющих цепочки x [11]. Пример неразмеченного дерева составляющих представлен на рисунке 1.3.

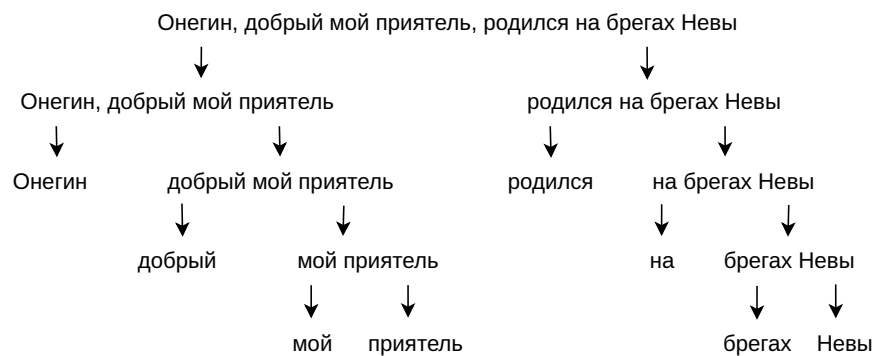


Рисунок 1.3 — Неразмеченное дерево составляющих для предложения на русском языке

Размеченной системой составляющих называют упорядоченную тройку:

$$\langle C, W, \varphi \rangle, \quad (1.2)$$

где C — система составляющих,
 W — конечное множество меток,

φ — разметка, отображение C в множество всех непустых подмножеств W .

Таким образом дерево составляющих представляет собой дерево, в котором в листьях находятся слова предложения, поддеревья соответствуют входящим в предложение синтаксическим конструкциям, а дуги выражают отношение вложенности конструкций [12]. Пример размеченного дерева составляющих представлен на рисунке 1.4.

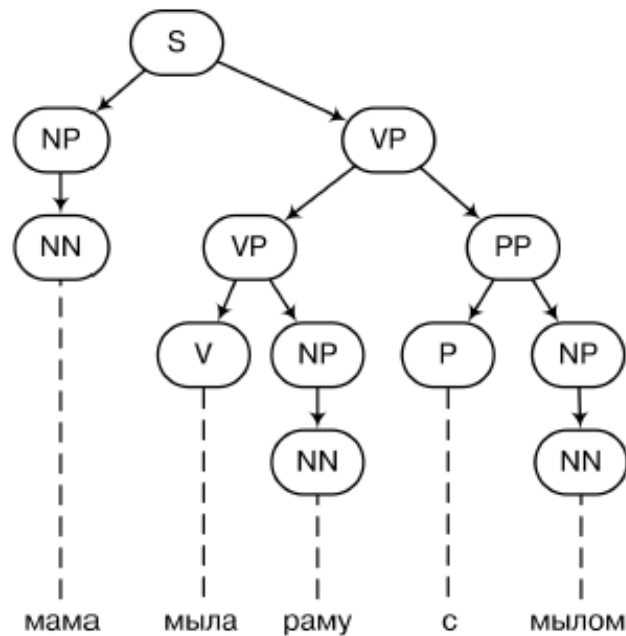


Рисунок 1.4 — Размеченное дерево составляющих для предложения на русском языке

К преимуществам системы составляющих относят [16, 17, 18, 19]:

- явную разметку фразовых категорий и синтаксических единиц;
- учет грамматически правильного порядка слов;
- построение грамматики для работы алгоритма на основе типов составляющих, что является более общим подходом, чем части речи;
- возможность различить адъюнкты и аргументы при предикате;
- возможность описать правила сочинения составляющих;
- применимость построенных деревьев для дальнейшего семантического анализа;
- возможность использования полученных синтаксических единиц и групп для прикладных задач.

Основными недостатками систем составляющих являются [12, 16, 18, 19]:

- возможность построения нескольких деревьев составляющих для одного предложения;
- увеличение количества правил в грамматике с ослаблением требований к порядку слов;
- необходимость ввода дополнительных правил и «синтетических» составляющих для учета небинарных правил и непроективности;
- высокая трудоемкость ручной разметки;

— необходимость ручного определения отношения «главное–зависимое».

1.2.3 Системы синтаксических групп

Пусть $E_1, E_2 \subseteq P$, некоторого непустого конечного множества, на котором определено отношение линейного порядка. E_1 и E_2 зацепляются, если $E_1 \cap E_2 = \emptyset$ и E_1 не содержится ни в какой циклической компоненте множества $P \setminus E_2$ [11].

Пусть также C — некоторое множество непустых подмножеств P . Синтаксическими группами (СГ) называются элементы множества C . Тривиальными СГ называются одноэлементные СГ и P [11].

Если E_1, E_2 — две СГ, такие что $E_1 \subset E_2$ и не существует СГ E_3 , такой, что $E_1 \subset E_3 \subset E_2$, то СГ E_1 непосредственно вложена в E_2 .

Системой синтаксических групп называется граф $\langle C, \rightarrow \rangle$, для которого выполняются следующие условия:

- 1) C содержит P и все одноэлементные подмножества P ;
- 2) если $E_1, E_2 \in C$ то либо $E_1 \cap E_2 = \varphi$, либо $E_1 \subseteq E_2$, либо $E_2 \subseteq E_1$;
- 3) если $E_1, E_2 \in C$, то E_1 и E_2 не зацепляются;
- 4) если $E_1 \rightarrow E_2$, то E_1 и E_2 непосредственно вложены в одну и ту же СГ;
- 5) СГ E не может зависеть от самой себя;
- 6) если $E_1 \rightarrow E_2$ и $E_2 \rightarrow E_1$, то $E_1 = E_2$;
- 7) если $E_1 \rightarrow E_2$ и E — произвольная СГ, то множества E и $E_1 \cup E_2$ не зацепляются;
- 8) если $E_1 \rightarrow E_2, E_3 \rightarrow E_4$, то множества $E_1 \cup E_2$ и $E_3 \cup E_4$ не зацепляются.

Размеченной системой синтаксических групп называется упорядоченная шестерка 1.3:

$$\langle C, \rightarrow, W, Z, \varphi, \psi \rangle, \quad (1.3)$$

где $\langle C, \rightarrow \rangle$ — система синтаксических групп,

W — конечное множество меток СГ;

Z — конечное множество меток при стрелках (типами синтаксических связей);

φ — отображение C в множество всех подмножеств W ;

ψ — отображение множества дуг графа $\langle C, \rightarrow \rangle$ в множество всех подмножеств Z .

В соответствии с [11] неразмеченная система синтаксических групп представлена на рисунке 1.5.

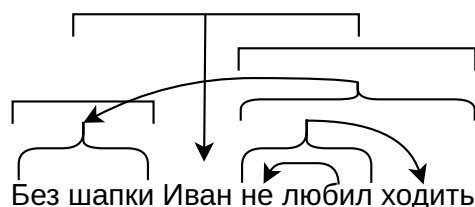


Рисунок 1.5 — Неразмеченное дерево системы синтаксических групп для предложения на русском языке

Система синтаксических групп разработана и описана А. В. Гладким в работах [11, 20], как альтернатива системам составляющих и зависимостей, объединяющая используемые в них подходы, и рассматривается в более поздних работах [16, 21]. Основными преимуществами систем СГ в этих работах выделяются:

- возможность задать одновременно и вложенность групп, и иерархию подчинения отдельных словоформ;
- возможность задать разрывные и непроективные группы без введения дополнительных конструкций и правил;
- отсутствие необходимости адаптировать данную систему к русскому языку: А. В. Гладкий изначально разрабатывал ее в приложении к русскому языку.

Недостатками системы СГ следующие [11, 16, 20, 21]:

- множество групп получается в ходе построения системы и не является счетным;
- для одного предложения возможно получить несколько альтернативных систем СГ;
- лингвистические правила для построения систем синтаксических групп должны совмещать правила построения систем зависимостей;
- ССГ в том виде, в котором ее описал А. В. Гладкий не покрывает в полной мере сочинительных конструкций;
- системы СГ не получили широкого распространения в корпусной лингвистике русского языка.

1.2.4 Сравнение систем синтаксической разметки

Для сравнения систем синтаксической разметки были выделены следующие критерии:

- возможность выделения фразовых категорий;
- поддержка непроективных предложений;
- возможность построения альтернативных деревьев разбора предложения;
- использование системы в качестве источника для порождающей грамматики;
- наличие общедоступных корпусов русского языка с разметкой данным видом систем.

Сравнение систем синтаксической разметки по приведенным критериям представлено в таблице 1.1.

Таблица 1.1 — Сравнение систем синтаксической разметки

	Системы зави- сисмостей	Системы составля- ющих	Системы синтакси- ческих групп
Выделение фразовых категорий	Нет	Да	Да
Поддержка непроек- тивных предложений	Нет	Да	Да
Возможность построе- ния нескольких дере- вьев разбора	Нет	Да	Да
Использование систе- мы как порождающей грамматики	Да	Да	Нет
Общедоступные кор- пуса с разметкой та- ким видом систем	Да	Да	Нет

Таким образом, системы составляющих являются наиболее предпочтительными для реализации, как наиболее полно отражающие синтаксические структуры русского языка и в то же время, являющиеся в достаточной степени распространенными в отечественной корпусной лингвистике [16]. В [7, 17] отмечается, что хотя общедоступные корпуса русского языка с разметкой систем составляющих распространены гораздо меньше, чем общедоступные корпуса с разметкой систем зависимостей, разметка систем составляющих, создание корпусов и средств автоматизации выполнения данного вида разметки является важным для лингвистических исследований.

1.3 Методы построения систем составляющих

1.3.1 Алгоритм Кока–Янгера–Касами

В [18] описан алгоритм Кока—Янгера—Касами (СҮК) построения дерева составляющих для контекстно-свободной грамматики. Перед разбором грамматику приводят к нормальной форме Хомского (НФХ): правила имеют вид $A \rightarrow BC$, где в правой части находятся два нетерминала, или $A \rightarrow w$, где w — терминал [18]. Любую контекстно-свободную грамматику можно преобразовать в слабо эквивалентную грамматику в НФХ; при этом структура деревьев разбора сохраняется с точностью до введения вспомогательных нетерминалов [18].

Алгоритм использует подход динамического программирования. Для предложения из n словоформ заполняют верхнетреугольную часть таблицы dp размером $(n+1) \times (n+1)$. Индексы $i, j = \overline{0, n}$ соответствуют позициям *между* словами (от нуля до n); ячейка $dp_{i,j}$ при $i < j$ хранит множество нетерминалов, порождающих подстроку слов с i -й по j -ю позицию. На «диагонали» длины 1, то есть в $dp_{i,i+1}$, после чтения i -го слова помещают нетерминалы (и при необходимости

частеречные категории) по лексическим правилам $A \rightarrow w_i$.

Поскольку в НФХ у каждой внутренней вершины дерева составляющих ровно два потомка, нетерминал A может появиться в $dp_{i,j}$ только при некотором разбиении $i < k < j$ и правиле $A \rightarrow BC$, где $B \in dp_{i,k}$ и $C \in dp_{k,j}$. Формально для всех $i < k < j$ выполняется выражение 1.4.

$$dp_{i,j} = dp_{i,j} \cup \{A \mid A \rightarrow BC \in P, B \in dp_{i,k}, C \in dp_{k,j}\}. \quad (1.4)$$

Пример заполнения таблицы dp представлен на рисунке 1.6.

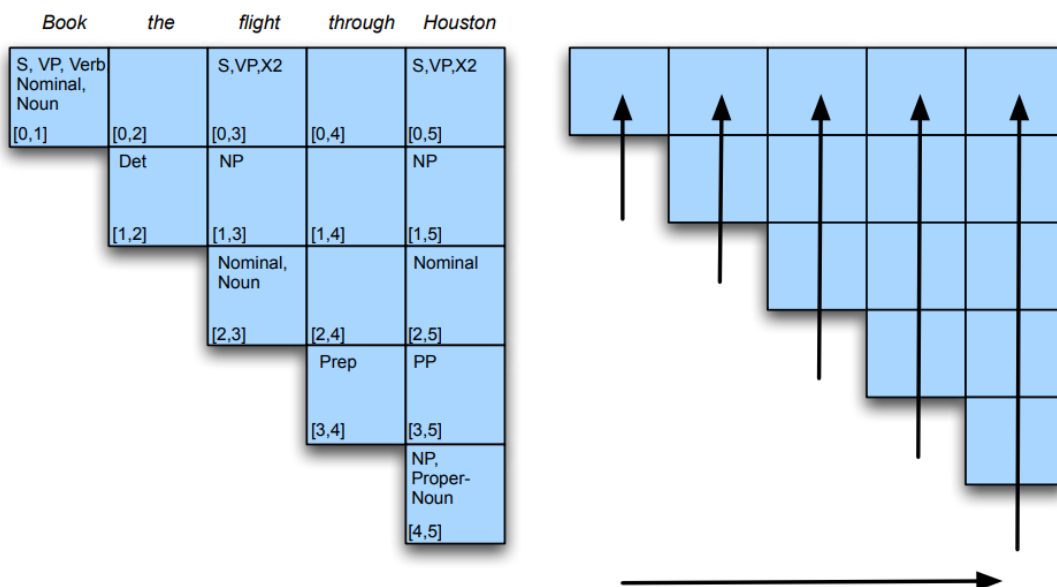


Рисунок 1.6 — Пример заполнения таблицы для предложения «Book the flight through Houston»

Заполнение таблицы dp производится слева направо, снизу-вверх, к моменту вычисления $dp_{i,j}$ известны значения всех $dp_{l,j}$, $l < i$ и $dp_{i,m}$, $m < j$, соответственно могут быть проверены все возможные разбиения интервала $[i,j]$. Варианты разбора входного предложения, таким образом, находятся в $dp_{0,n}$. Выбор верного варианта разбора среди представленных в $dp_{0,n}$ осуществляется вручную или предобученным классификатором.

Для построения дерева составляющих по заполненной таблице dp при добавлении A в $dp_{i,j}$ из правила $A \rightarrow BC$ и разбиения k сохраняют указатели на $dp_{i,k}$ и $dp_{k,j}$ и использованное правило. Восстановление дерева — рекурсивный обход от выбранного $S \in dp_{0,n}$. При нескольких разборах выбор одного дерева выносится за рамки классического СУК и может выполняться вручную, по дополнительным ограничениям грамматики или с помощью предобученной модели, оценивающей правдоподобность конститuent [18].

Вычислительная сложность распознавания для грамматики фиксированного размера составляет $O(n^3)$ при длине предложения n [22].

1.3.2 Алгоритм Эрли

Алгоритм Эрли [18, 23] относится к методам синтаксического разбора контекстно-свободных грамматик также с использованием подхода динамического программирования.

Для предложения из n словоформ данный алгоритм формирует массив dp множеств состояний размера $n + 1$. dp_i соответствует множеству возможных состояний частично построенного дерева зависимостей после обработки i словоформ. При этом в ходе работы, алгоритм совершает единственный обход предложения слева-направо, не обрабатывая повторно уже полученные составляющие.

Состояние описывается тройкой вида 1.5:

$$\langle r, p, [i, j] \rangle, \quad (1.5)$$

где r — правило контекстно-свободной грамматики вида $A \rightarrow (X_1, X_2, \dots, X_m)$;

$p \in \overline{0, m}$ — позиция отметки в правой части правила r ;

$[i, j]$, $0 \leq i \leq j \leq n$ — границы фрагмента исходного предложения, соответствующие правой части правила r , расположенной до метки p .

Начальное состояние разбора описывается тройкой 1.6 и помещается в множество состояний dp_0 до начала работы алгоритма.

$$\langle \gamma \rightarrow S, 0, [0, 0] \rangle, \quad (1.6)$$

где γ — вспомогательный нетерминал;

S — стартовый символ грамматики.

Предложение из n словоформ считается успешно разобранным в случае, если тройка 1.7 находится в множестве состояний dp_n .

$$\langle \gamma \rightarrow S, 1, [0, n] \rangle \quad (1.7)$$

Обработка префикса предложения из i словоформ заключается в обходе каждого состояния множества dp_i и применения ровно одной из трёх нижеописанных операций. Полученные в результате применения операций состояния добавляются в dp_i или dp_{i+1} .

Обработку ведут слева направо: для каждого $i = \overline{0, n}$ последовательно рассматривают состояния в $Chart_i$ (без возврата к ранее обработанным позициям). В зависимости от вида состояния применяют один из трёх операторов. **Predictor**: если справа от отметки стоит нетерминал B , в $Chart_i$ добавляют все состояния $(B \rightarrow \bullet\gamma; [i, i])$ по правилам $B \rightarrow \gamma$ грамматики — это нисходящие ожидания. **Scanner**: если справа от отметки терминал (часть речи) t и текущее слово w_i с ним согласуется, в $Chart_{i+1}$ добавляют состояние с отметкой, сдвинутой через w_i . **Completer**: если правило полностью разобрано, $(B \rightarrow \gamma\bullet; [j, i])$, то для каждого состояния $(A \rightarrow \alpha\bullet B\beta; [l, j])$ в $Chart_j$ добавляют в $Chart_i$ состояние $(A \rightarrow \alpha B\bullet\beta; [l, i])$ — найденный

конституент B «закрывает» ожидание в другом правиле. Одинаковые состояния в множество не дублируют.

Разбор успешен, если в $Chart_n$ есть завершённое состояние $(\gamma \rightarrow S\bullet; [0, n])$. Как и в СΥК, чарт неявно хранит все допустимые разборы: каждое поддерево фиксируется один раз и используется во всех деревьях, которым оно нужно. Для получения конкретного дерева составляющих оператор *Completer* дополняют сохранением ссылок на породившие состояния (по аналогии с обратными указателями в таблице dp).

В худшем случае время работы алгоритма Эрли оценивается как $O(n^3)$ при длине предложения n ; для многих неоднозначных грамматик на практике наблюдается поведение ближе к $O(n^2)$ [18, 23]. Для отдельных классов грамматик (например, $LR(k)$) возможна линейная сложность. СΥК и алгоритм Эрли решают одни и те же задачи мемоизации поддеревьев при разборе: СΥК удобен при фиксированной грамматике в НФХ и явной таблице конституент, алгоритм Эрли — при произвольной КС-грамматике и пошаговой обработке предложения слева направо.

1.4 Постановка задачи

Для реализации метода построения синтаксических деревьев ограниченного естественного русского языка необходимо определить:

- определить формат входных и выходных данных;
- определить необходимые морфологические характеристики словоформ;
- определить правила грамматики применяемые в ограниченном русском языке;
- определить типы согласования применяемые в ограниченном русском языке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бутенко Ю. И. Модели и методы автоматической обработки научно-технических текстов в параллельном корпусе. Докторская диссертация: Федеральное государственное автономное образовательное учреждение высшего образования «Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)» (МГТУ им. Н. Э. Баумана). Москва, 2025.
2. Мухамедшин Д. Р., Сулейманов Д. Ш. Система корпус-менеджер: архитектура и модели корпусных данных // Программные продукты и системы. 2018. Т. 31, № 4. С. 653–658.
3. Добровольский Д. О., Зацман И. М. Интеграция электронного словаря с текстами параллельного корпуса: новый теоретический подход // Системы и средства информатики. 2025. Т. 35, № 1. С. 111–124.
4. Файзиева С. А. Теоретические основы и значимость электронных корпусов параллельных текстов в переводческой деятельности // INTERNATIONAL SCIENTIFIC JOURNAL: LEARNING AND TEACHING. 2025. Т. 2, № 3. С. 71–74.
5. Авагян Н. А. Параллельный корпус научно-технических текстов как инструмент переводчика // Актуальные проблемы лингвистики и лингводидактики в неязыковом вузе: сборник материалов 4-ой Международной научно-практической конференции, Москва, 16 декабря 2020 года: в 2 т. Т. 1. МГТУ им. Н.Э. Баумана, 2021. С. 194–199.
6. Естественно-языковые интерфейсы интеллектуальных вопросно-ответных систем / В. А. Житко, В. Н. Вяльцев, Ю. С. Гецевич [и др.]. 2011.
7. Захаров В. П., Богданова С. Ю. Корпусная лингвистика: учебник для студентов гуманитарных вузов. Иркутск: ИГЛУ, 2011. 161 с., ISBN 978-5-88267-316-0.
8. Leech Geoffrey. Introducing corpus annotation. 1997.
9. Копотев М. В. Введение в корпусную лингвистику. Прага: Animedia Company, 2014. 231 с., ISBN: 978-80-7499-067-0.
10. Treebanks: Building and Using Parsed Corpora / под ред. Anne Abeillé. Dordrecht: Springer, 2003. Т. 20 из *Text, Speech and Language Technology*.
11. Гладкий А. В. Синтаксические структуры естественного языка в автоматизированных системах общения. Проблемы искусственного интеллекта. М.: Наука. Главная редакция физико-математической литературы, 1985. с. 144.

12. Автоматическая обработка текстов на естественном языке и анализ данных / Е. И. Большакова, К. В. Воронцов, Н. Э. Ефремова [и др.]. М.: Изд-во НИУ ВШЭ, 2017. с. 269.
13. Nivre Joakim. An Efficient Algorithm for Projective Dependency Parsing // Proceedings of the 7th Conference on Natural Language Learning at HLT-NAACL 2003. Edmonton, Canada: Association for Computational Linguistics, 2003. С. 1–8. URL: <https://aclanthology.org/W03-3017/>.
14. MaltParser: A Language-Independent System for Data-Driven Dependency Parsing / Joakim Nivre, Johan Hall, Jens Nilsson [и др.] // Natural Language Engineering. 2007. Т. 13, № 2. С. 95–135.
15. Киселёв М. В., Федосеева Д. В. Синтаксический парсер русского языка LParus компании Megaruter Intelligence // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог». Москва: РГГУ, 2017.
16. Тестелец Я. Г. Введение в общий синтаксис. М.: Российский государственный гуманитарный университет, 2001.
17. Гращенков П. В. RuConst: синтаксический корпус русского языка с разметкой по непосредственным составляющим // Вестник Московского университета. Серия 9. Филология. 2024. № 3. С. 94–112.
18. Jurafsky Daniel, Martin James H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. Third draft изд. 2026. Draft of January 6, 2026.
19. Полетаев А. Ю., Парамонов И. В., Бойчук Е. И. Алгоритм построения дерева синтаксических единиц русскоязычного предложения по дереву синтаксических связей // Информатика и автоматизация. 2023. Т. 22, № 6. С. 1323–1353.
20. Гладкий А. В. К процедуре построения систем синтаксических групп // Московский лингвистический журнал. М., 1998. Т. 4. С. 32–45.
21. Коротаев Н. А. Синтаксические группы А. В. Гладкого: анализ конструкций с сочинением // Вестник Российского государственного гуманитарного университета. Серия: Литературоведение. Языкознание. Культурология. М., 2013. № 8. С. 17–36. Фрагмент дипломной работы (РГГУ, 2002); в ряде библиографий указаны с. 16–36.
22. Makarov V. Cocke–Younger–Kasami–Schwartz–Zippel algorithm and relatives. 2022.
23. Earley Jay. An Efficient Context-Free Parsing Algorithm // Communications of the ACM. 1970. Т. 13, № 2. С. 94–104.

ПРИЛОЖЕНИЕ А

Презентация к научно-исследовательской работе на 8 слайдах.